# COMP4601 Project B Demonstration Plan

## Brief Objective

This demonstration shows the end-to-end operation of a hardware-accelerated LeNet-5 CNN on the AMD Kria KV260 FPGA, and quantifies the gains achieved through a three-stage optimisation methodology. Observers will be able to verify both correct output (classification results) and measured performance (latency and profiling data) for the software baseline and each acceleration step, using the same network and the same workload throughout.

## Materials Checklist

Board and connections:

- Kria KV260 powered on and booted
- SSH access confirmed to 10.42.0.132 (petalinux@)
- Ethernet cable connected between laptop and board

Files on the board (verify with ls before the demo):

- lenet5_host binary in /home/petalinux/
- lenet5.bin bitstream (full-accel, FC ×16 build) in /home/petalinux/
- Pre-built V0, V1, V2 .bin files if running profiling comparison on-board, or pre-captured terminal output screenshots as fallback

Files on the demo laptop:

- draw_demo_app.py present and tested (Python + Pillow + tkinter installed)
- SSH key or password ready so SCP doesn't prompt mid-demo

Pre-opened on screen before observers arrive:

- draw_demo_app.py running and showing the canvas
- Terminal window with SSH session to the board already active

# Part 1: Live Inference

**Goal:** Show the HW and SW paths producing matching predictions on a hand drawn digit, and show the latency difference between them.

## Setup (pre-demo)

- Board powered on, bitstream loaded (sudo xmutil loadapp lenet5), SSH verified.
- draw_demo_app.py open and running on the demo laptop (connected to board at 10.42.0.132).
- Board running the **full-accel** build (lenet5_full_accel).

## Steps

1. **Draw a digit** on the canvas (e.g. "3", "7"). Show the raw canvas on screen.
2. **Click "Classify Hardware".**
    - App SCPs the 28×28 pixel text file to the board and calls lenet5_host -x lenet5.bin -i live_digit.txt.
    - Terminal output appears in the Hardware panel: predicted digit, confidence bar chart, and FPGA inference time (µs).
    - *Expected: correct prediction, ~300 µs FPGA time reported.*
3. **Click "Classify Software"**.
    - App calls lenet5_host -x lenet5.bin -s live_digit.txt (ARM software path on the same binary).
    - Terminal output appears in the Software panel: same predicted digit, same confidence scores, but ARM inference time (~20000 µs baseline order).
    - *Both panels should agree on the digit showing demonstrating result correctness.*
4. **Repeat with a few more digits**.

**Note:** Single image runs are always a 'cold start'. They have a new process opening the device and loading the kernel just for one image, so they appear to run a bit slower than our batch-average number, which is measured across 100 images once everything's already set up. Both simply measure slightly different things: one-shot latency vs. sustained throughput.

# Part 2: Profiling Comparison

**Goal:** Walk through terminal output from all three versions of the full LeNet-5 implementation to show measurable, stepwise speedup and unchanged accuracy.

## The Three Versions

| Version | Description | Latency | Speedup (compared to sw) |
|---|---|---|---|
| **V0** lenet5_unoptimised | Baseline: sequential, no HLS pragmas | ~6700 µs | ~1.72× |
| **V1** lenet5_conv_accel | Conv layers accelerated (fixed-point + adder tree) | ~2300 µs | ~7.77× |
| **V2** lenet5_full_accel | Conv + FC accelerated (FC unroll ×16) **best deployable** | ~168 µs | ~114.87x |

## Steps

1. **Run V0 (baseline) on 100 MNIST test images:**

   sudo xmutil unloadapp && sudo xmutil loadapp lenet5 && ./lenet5_host -x lenet5.bin
   - **100% accuracy**, ~6700 µs per image, II=9 bottleneck on conv and FC visible in HLS report.

2. **Run V1 (conv-only acceleration) on 100 images:**

   sudo xmutil unloadapp && sudo xmutil loadapp lenet5ca && ./lenet5ca_host -x lenet5ca.bin
   - **100% accuracy maintained**, latency drops to ~2300 µs (7.77×).
   - Highlight: accelerating conv alone capped the gain because FC1 was still the bottleneck.

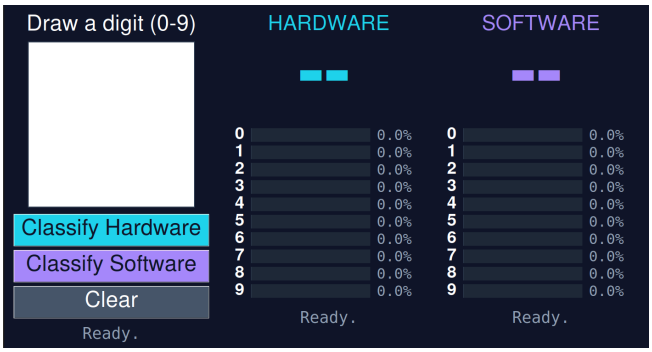
3. **Run V2 (full acceleration, FC ×16) on 100 images:**

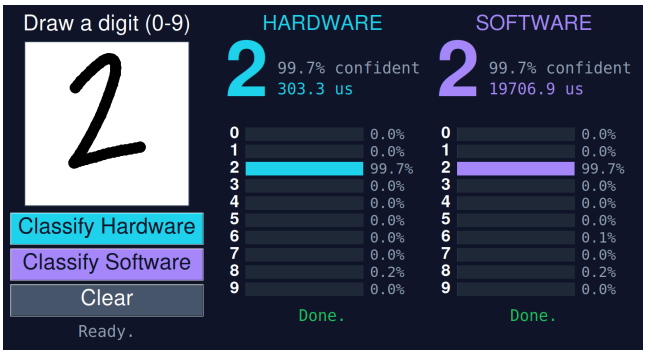
   sudo xmutil unloadapp && sudo xmutil loadapp lenet5fa && ./lenet5fa_host -x lenet5fa.bin
   - **100% accuracy maintained**, latency drops to ~168.5 µs (114.87×).
   - Highlight: spending the remaining LUT budget on the FC bottleneck, not more conv parallelism, was the right decision. V3 (max conv unroll) used 109% LUT and couldn't be placed, for only a 1.2× gain over V2.

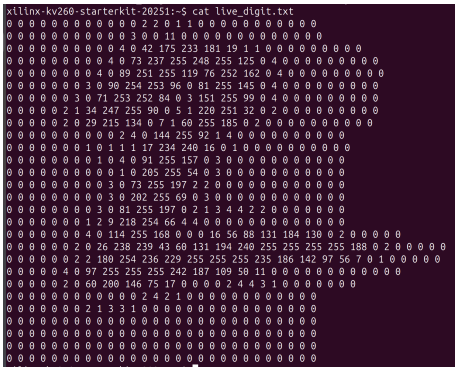
## Success Criteria

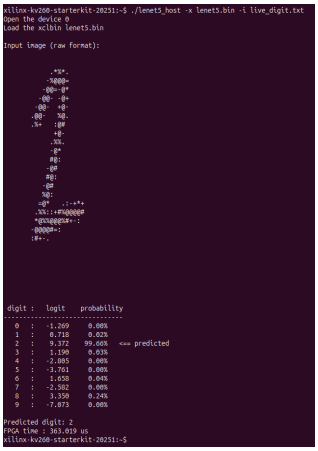
The demonstration passes if observers can confirm:

1. The draw-demo app produces the same predicted digit on both the Hardware and Software panels for every digit drawn, results must agree, not just be plausible.
2. All three builds (V0, V1, V2) run to completion on the 100-image MNIST batch without errors, each printing 100% accuracy.
3. The latency numbers from terminal output match the expected progression: ~6700 µs → ~2300 µs → ~168 µs.




```
  image  97: predicted 7, actual 7
  image  98: predicted 6, actual 6
  image  99: predicted 9, actual 9

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---
FPGA time : 6705.935 us/image
ARM  time : 11547.589 us/image
Speedup   : 1.72x
xilinx-kv260-starterkit-20251:~$
```

```
  image  97: predicted 7, actual 7
  image  98: predicted 6, actual 6
  image  99: predicted 9, actual 9

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---
FPGA time : 2356.470 us/image
ARM  time : 18301.227 us/image
Speedup   : 7.77x
xilinx-kv260-starterkit-20251:~$
```

```
  image  97: predicted 7, actual 7
  image  98: predicted 6, actual 6
  image  99: predicted 9, actual 9

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---
FPGA time : 168.820 us/image
ARM  time : 19392.635 us/image
Speedup   : 114.87x
xilinx-kv260-starterkit-20251:~$
```